

第十五周作业-2

提高作业

2.1 Hough 与 YOLO 感知方案对比

比较 Hough 圆检测和 YOLO 检测在足球识别任务中的特点。
完成内容：

- 说明 Hough 圆检测的输入、输出和基本思路；
- 说明 YOLO 检测的输入、输出和基本思路；
- 比较两者对光照、背景、遮挡、球颜色和运行速度的敏感性；
- 说明为什么 YOLO Demo 中需要 fallback；
- 分析在 VMware 虚拟机中运行 YOLO 可能遇到的性能问题，并给出解决思路；
- 如果让你应用 YOLO 进行目标检测，简要设计一套从数据采集、模型训练到部署运行的方案。

提交内容：

- Hough 与 YOLO 的对比表；
- 两种方案的优缺点分析；
- YOLO 性能分析与方案设计。

==Hough 圆检测 vs YOLO 检测：足球识别对比分析: ==

一、Hough 圆检测

输入

项目	内容
输入话题	<code>/robot1/camera/image_raw</code> （ <code>sensor_msgs/Image</code> ）
处理分辨率	原图 640×480
关键参数	HSV 阈值、dp、param1、param2、min/max_radius

输出

```
/robot1/vision/detections → Float32MultiArray
[x_err, y_err, w_norm, h_norm, 1.0, 0.0]
```

置信度固定为 1.0，class_id 固定为 0.0（无类别概念）。

基本思路

```
原图（BGR）
|
▼ cvtColor → HSV 色彩空间
|
▼ inRange([5,80,80] ~ [30,255,255]) → 橙色掩膜
|
▼ GaussianBlur(7×7) → 平滑边缘、减少噪点
|
▼ HoughCircles(HOUGH_GRADIENT)
|   dp=1.2      ← 分辨率因子（越大越粗糙）
|   param1=80   ← Canny 高阈值（边缘检测灵敏度）
|   param2=12   ← 圆心累加器阈值（越小越容易误检）
|   minRadius=4, maxRadius=80
|
▼ 多圆候选打分：score = mask_mean × radius
   选最优圆（最大且最橙的圆）
|
▼ 归一化圆心坐标 → 发布 vision/detections
```

本质：这是一种基于颜色 + 几何形状的规则算法，不需要任何训练数据，所有"知识"都编码在 HSV 阈值和 HoughCircles 参数中。

二、YOLO 检测

输入

项目	内容
输入话题	<code>/robot1/camera/image_raw</code>
推理分辨率	缩放到 <code>imgsz=320×320</code> （配置项）

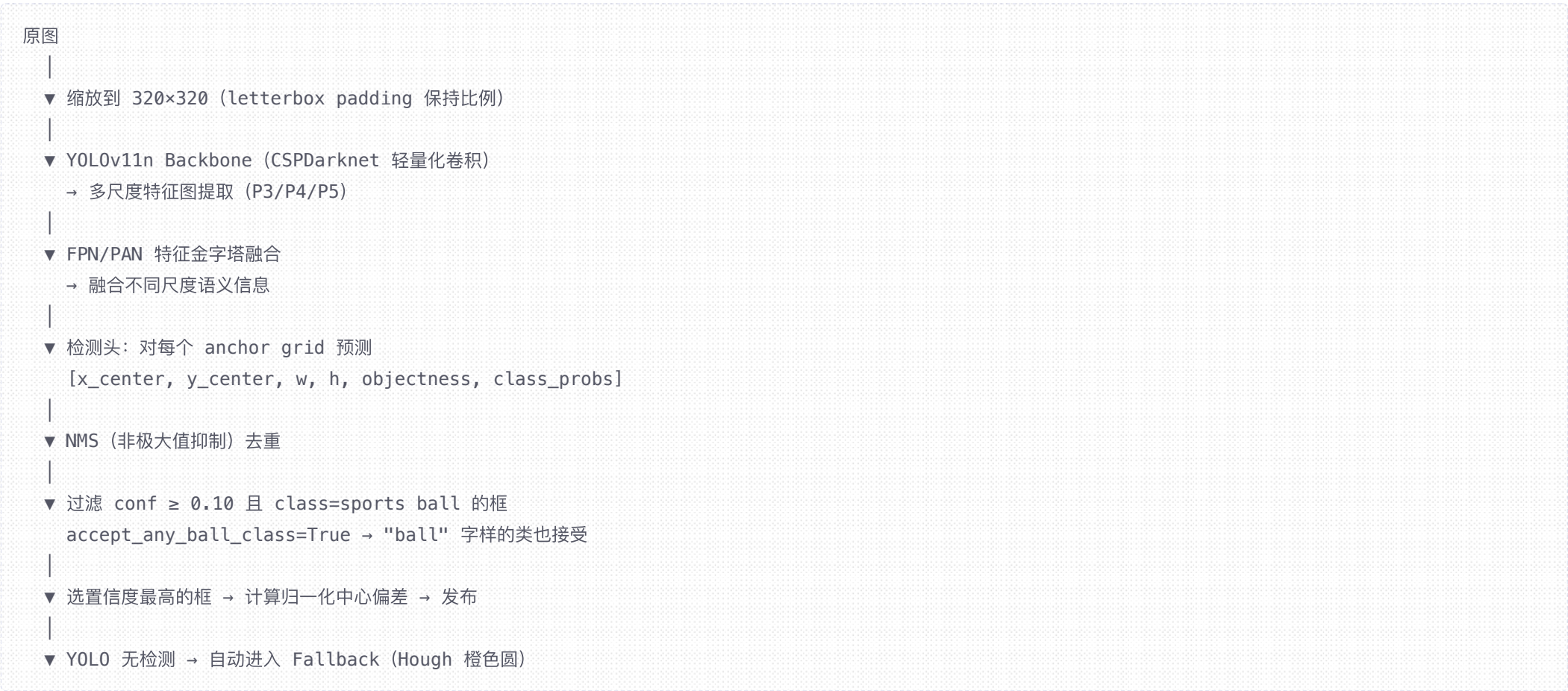
项目	内容第十五周作业-2
模型	YOLOv11 nano (yolo11n.pt , 5.4 MB)
置信度阈值	0.10 (极低, 宁可多检测)
目标类别	sports ball (COCO 第 37 类)

输出

```
/robot1/vision/detections → Float32MultiArray
[x_err, y_err, w_norm, h_norm, conf, cls_id]

同时发布 /robot1/yolo/debug_image
    标注: 绿色框 = YOLO 检测, 橙色框 = FALLBACK
```

基本思路



本质：这是一种端到端深度学习方法，通过在大规模数据集（COCO）上训练，学习到球的形状、纹理、颜色等综合特征的表达。

三、两者的敏感性对比

对比维度	Hough 圆检测	YOLO 检测
光照变化	△ 敏感。强光/弱光会改变 HSV 值，导致掩膜漏检或大量噪点。早晨/傍晚或室内灯光不均匀时误检率显著升高	✅ 较鲁棒。神经网络在训练数据中见过各种光照条件，对亮度变化有一定泛化
背景复杂度	△ 敏感。草地上若有橙色标记线、广告牌等，会被误认为球	✅ 较好。YOLO 理解语义，能区分球与橙色背景物体
遮挡	❌ 脆弱。圆形不完整时 HoughCircles 无法检测，半遮挡的球容易丢失	✅ 较好。训练数据中包含遮挡样本，能检测到部分可见的球
球颜色变化	❌ 完全依赖 HSV 阈值。换成白色或花纹球则彻底失效，必须重新调参	✅ 能识别多种外观的球，因训练集覆盖不同样式
球大小变化	△ 依赖 min/max_radius 参数范围，超出范围则漏检	✅ FPN 多尺度设计，对远近大小变化更鲁棒
运行速度	✅ 极快。CPU 纯算法，15 Hz，延迟 < 20ms	△ 慢。CPU 推理约 2 Hz，GPU 可达 30+ Hz
计算资源	✅ 极低，CPU < 5%	❌ 高，CPU 推理时单核近 100%，需 GPU 加速
误检率	△ 高（橙色物体都可能误检）	✅ 低（语义级别识别）
参数调试	△ 需要手动调 HSV 阈值，场地/光线变时必须重调	✅ 基本无需调参，换场景只需重新训练
训练数据	✅ 无需数据	△ 需要标注数据集
可解释性	✅ 每步可视化，易于调试	△ 黑盒，失败原因难排查

四、为什么 YOLO Demo 中需要 Fallback

本项目的具体原因

在 Gazebo 仿真环境中使用 YOLO 有三个根本矛盾：

1. 训练分布 vs 仿真图像的域差距（Domain Gap）

YOLOv11n 训练集: COCO (真实世界照片)

↓ 球的外观: 真实皮革纹理、复杂光影、自然背景

Gazebo 渲染的球: 平面橙色纯色圆形、无纹理、光照均匀

2 / 9

↓

第十五周作业-2

YOLO "没见过"这种过于简单的球 → 置信度极低 → 低于阈值不输出

2. 置信度阈值已经设到极低（0.10）仍可能漏检
- 代码中 **confidence = 0.10** 已经是非常宽松的阈值，说明 YOLO 对仿真球的置信度可能连 0.10 都达不到。
3. 推理频率仅 2 Hz，帧间隔长

publish_rate_limit: **2.0** # 两次推理之间至少 0.5s

若在 0.5s 间隔内球移动到了视野边缘，该帧 YOLO 未检测 → 此时没有 Fallback 则视觉链路断裂 → FSM 切换 SEARCH_BALL → 机器人原地转圈。

Fallback 机制的代码逻辑

```
# yolo_detector_node.py _image_callback()
best, detections = self._run_yolo(image)      # 先跑 YOLO

if best is not None:                          # YOLO 成功
    self._publish_detection(best, ...)
    self._draw_detection(debug, best, (0,255,0), "YOLO")
    return

# YOLO 失败 → 走 Fallback
fallback = self._fallback_circle(image)       # Hough 橙色圆
if fallback is not None:
    self._publish_detection(fallback, ...)
    self._draw_detection(debug, fallback, (0,200,255), "FALLBACK")
```

结论：Fallback 是用 Hough 来弥补 **YOLO** 在仿真场景下的泛化不足，保证视觉链路不中断，是工程上的鲁棒性设计。

五、VMware 虚拟机运行 YOLO 的性能问题与解决思路

问题分析

性能瓶颈	原因	影响
无 GPU 直通	VMware Workstation 默认不支持 CUDA GPU 直通；即使开启 3D 加速也无法使用 CUDA	YOLO 强制 CPU 推理，速度慢 5~20 倍
CPU 资源争抢	虚拟机与宿主机共享物理核心，Gazebo + ROS + YOLO 同时运行时 CPU 饱和	推理延迟剧增，ROS 消息延迟累积
内存带宽受限	虚拟化层引入额外内存拷贝开销	numpy/tensor 数据搬运慢
时钟漂移	VMware 时钟精度差，ROS 时间戳不稳定	ROSTimeMovedBackwardsException ，IKWalk 步态相位重置
图像传输延迟	Gazebo 渲染 → /camera/image_raw → YOLO 节点，经过虚拟网络栈	端到端延迟 > 500ms

解决思路

优先级从高到低：

① 降低推理规模（最有效，代码层面）

imgsz: 320 → 160 推理速度提升 ~4x

confidence: 0.10 → 0.25 减少 NMS 计算量

publish_rate_limit: 2.0 → 1.0 降低推理频率

② 使用 ONNX Runtime 替代 PyTorch 推理

model.export(format='onnx')

使用 onnxruntime CPU 后端，比 PyTorch CPU 快 1.5~2x

OpenVINO 后端在 Intel CPU 上可再提速 2~3x

③ 分配更多 CPU 核心给虚拟机

VMware 设置：处理器数量 ≥ 4 核

开启"超线程"支持

④ 独立进程 + conda 环境（项目已支持）

使用 yolo_detector_conda.sh 在独立 conda 环境运行

避免 ROS Python 环境依赖冲突

⑤ 考虑 WSL2 替代 VMware（长期方案）

WSL2 支持 CUDA GPU 直通（需 Windows 11 + NVIDIA 驱动）

性能接近原生 Linux

⑥ 物理机双系统（最优）

原生 Ubuntu 20.04 + CUDA, GPU 推理速度 30+ Hz

本项目中 **Fallback** 的另一层价值：当 YOLO 在虚拟机中因为性能不足而推理极慢时，Hough（纯 CPU 算法，15 Hz）作为高频率补充，保证控制环路不断链。

六、YOLO 足球检测：从数据采集到部署的完整方案

6.1 整体流程图

数据采集 → 数据标注 → 数据增强 → 模型训练 → 评估验证 → 导出优化 → 部署集成 → 监控迭代

6.2 数据采集

目标：收集覆盖各种场景的足球图像，使模型泛化。

采集场景	具体内容
仿真截图	从 Gazebo <code>/robot1/camera/image_raw</code> 录制 rosbag，提取帧
真实场地	在不同光照（室内/室外/黄昏）、不同地板（草地/木地板/塑料场地）下拍摄
不同球	橙色足球、白色花纹球、小尺寸球（KidSize）、部分遮挡
负样本	不含球的背景图，防止误检

数量建议：≥ 2000 张（足球单类别任务，数据要求相对不高）。

6.3 数据标注

```
# 推荐工具: LabelImg (YOLO 格式直接输出)
pip install labelImg
labelImg ./images ./labels

# 每张图生成 .txt 文件，每行: class cx cy w h (归一化)
# 例: 0 0.512 0.634 0.087 0.112
```

标注规范：

- 只标注可见球（半遮挡也标，但完全遮挡不标）
- 边界框紧贴球外轮廓，不留白边

6.4 数据增强

利用 Ultralytics 内置增强（`data.yaml` + 训练参数）：

```
# 自动启用的增强（无需代码）
hsv_h: 0.015 # 色调扰动（应对光照变化）
hsv_s: 0.7    # 饱和度扰动
hsv_v: 0.4    # 亮度扰动
fliplr: 0.5   # 左右翻转
mosaic: 1.0   # 拼图增强（4张合1，学习小物体）
degrees: 10   # 随机旋转
scale: 0.5    # 随机缩放
```

针对本任务额外建议：**Mixup 仿真+真实图像**，缓解 domain gap。

6.5 模型训练

```
# data.yaml
path: ./soccer_dataset
train: images/train
val:   images/val
nc: 1
names: ['soccer_ball']
```

```
# 从 yolo11n 预训练权重 Fine-tune（迁移学习）
yolo train \
  model=yolo11n.pt \
  data=soccer_dataset/data.yaml \
  epochs=100 \
  imgsz=640 \
  batch=32 \
  lr0=0.001 \
  device=0 # GPU, CPU 改为 cpu
```

推荐策略：

场景	策略
数据少（< 500张）	冻结 backbone，只训练检测头 <code>freeze=10</code>
数据中等（500~2000）	全量 fine-tune，较小学习率
目标是仿真+真实	交替训练或域自适应（DA-YOLO）


```
yolo val model=runs/train/exp/weights/best.pt data=data.yaml imgsz=640
```

指标	含义	目标值
mAP@0.5	IoU>0.5 的平均精度	> 0.90
Precision	检测框中真正是球的比例	> 0.85
Recall	实际存在的球被检测到的比例	> 0.85
FPS (CPU)	CPU 推理帧率	> 10 Hz (imgsz=320)

6.7 模型导出与优化

```
# 导出为 ONNX (CPU 推理优化)
yolo export model=best.pt format=onnx opset=12 imgsz=320

# 导出为 OpenVINO (Intel CPU 最优)
yolo export model=best.pt format=openvino imgsz=320

# 导出为 TensorRT (NVIDIA GPU 最优)
yolo export model=best.pt format=engine imgsz=320 device=0
```

不同后端推理速度对比（640×480 图像，单帧）：

后端	硬件	推理时间
PyTorch CPU	i7-12700	~300ms
ONNX Runtime CPU	i7-12700	~150ms
OpenVINO	i7-12700	~80ms
TensorRT FP16	RTX 3060	~5ms

6.8 ROS 部署集成

```
# 替换 model_path 参数即可热插拔到现有节点
# soccer_hough_demo.launch 同层新建 soccer_custom_yolo.launch

<param name="model_path"
    value="$(find robot_soccer_integrated_demo)/models/soccer_ball_custom.onnx"/>
<param name="confidence" value="0.5"/> # 自训练模型可提高阈值
<param name="fallback_color_circle" value="false"/> # 自训练效果好可关闭
```

6.9 监控与迭代

部署后持续收集：

robot 视角的失败帧（YOLO 漏检 / 误检）

↓ 加入训练集重新标注

↓ Fine-tune（增量训练）

↓ 发布新版本权重

↓ 替换 model_path，热更新

主动难例挖掘（Hard Example Mining）：只收集 YOLO 输出 **FALLBACK** 标注的帧，这些正是模型表现最差的场景，优先标注加入训练集。

总结对比

	Hough 圆检测	YOLO 检测
适用场景	橙球、光照稳定	任意外观、复杂场景
速度	极快（15 Hz CPU）	慢（2 Hz CPU）
鲁棒性	低	高
部署难度	无需训练，即插即用	需要数据+训练
仿真效果	好（橙球完美契合）	差（需 Fallback）
真实场景	差（依赖颜色）	好（语义理解）
最佳组合	YOLO主 + Hough Fallback	

工程结论：在本仿真项目中 Hough 反而是更稳定的主力；而在真实足球机器人上，应以自训练 YOLO 为主检测器，Hough 仅作应急 Fallback。

2.2 双机器人 namespace 通信分析

运行或分析双机器人 Demo，理解 namespace 在多机器人系统中的作用。

参考命令：

5 / 9

```
roslaunch robot_soccer_integrated_demo two_robot_hough_demo.launch
```

可以使用以下命令查看两台机器人的关键话题：

```
rostopic echo /robot1/decision_staterostopic echo /robot1/cmd_velrostopic echo /robot2/goalie_staterostopic echo /robot2/cmd_vel
```

完成内容：

- 说明 `/robot1` 和 `/robot2` 的角色；
- 列出两台机器人的关键话题；
- 解释为什么多机器人系统必须使用 namespace；
- 画出双机器人通信结构图；
- 分析如果没有 namespace，可能出现哪些话题混淆或控制冲突。

提交内容：

- 双机器人关键话题表；
- 双机器人通信结构图；
- namespace 作用说明；
- 可能的话题混淆或控制冲突分析。

==双机器人 Demo：ROS Namespace 机制深度分析: ==

I 一、robot1 和 robot2 的角色

项目	robot1（进攻方）	robot2（守门方）
角色	Attacker（前锋）	Goalie（守门员）
初始位置	(0.0, 0.0)，朝向 0°	(3.8, 0.0)，朝向 180° (π rad)
感知方式	相机 → Hough 圆检测 → 视觉定位（视觉感知）	直接读 <code>/gazebo/model_states</code> （全局感知/作弊传感器）
决策节点	<code>soccer_demo_node</code> （FSM 6状态）	<code>goalie_node</code> （跟踪球 Y 坐标）
决策目标	找球 → 靠近 → 带球 → 射门 (4.5, 0.0)	跟踪球横向位置，守住 x=3.8 球门线
步态节点	<code>cmd_vel_adapter</code> + IKWalk	<code>cmd_vel_adapter</code> + IKWalk（同一套代码）
头部控制	<code>head_pitch_controller</code> （追踪球）	<code>head_pitch_controller</code> （仅保持姿态）
状态话题	<code>/robot1/decision_state</code>	<code>/robot2/goalie_state</code>

I 二、两台机器人的关键话题完整列表

I robot1（进攻方）话题

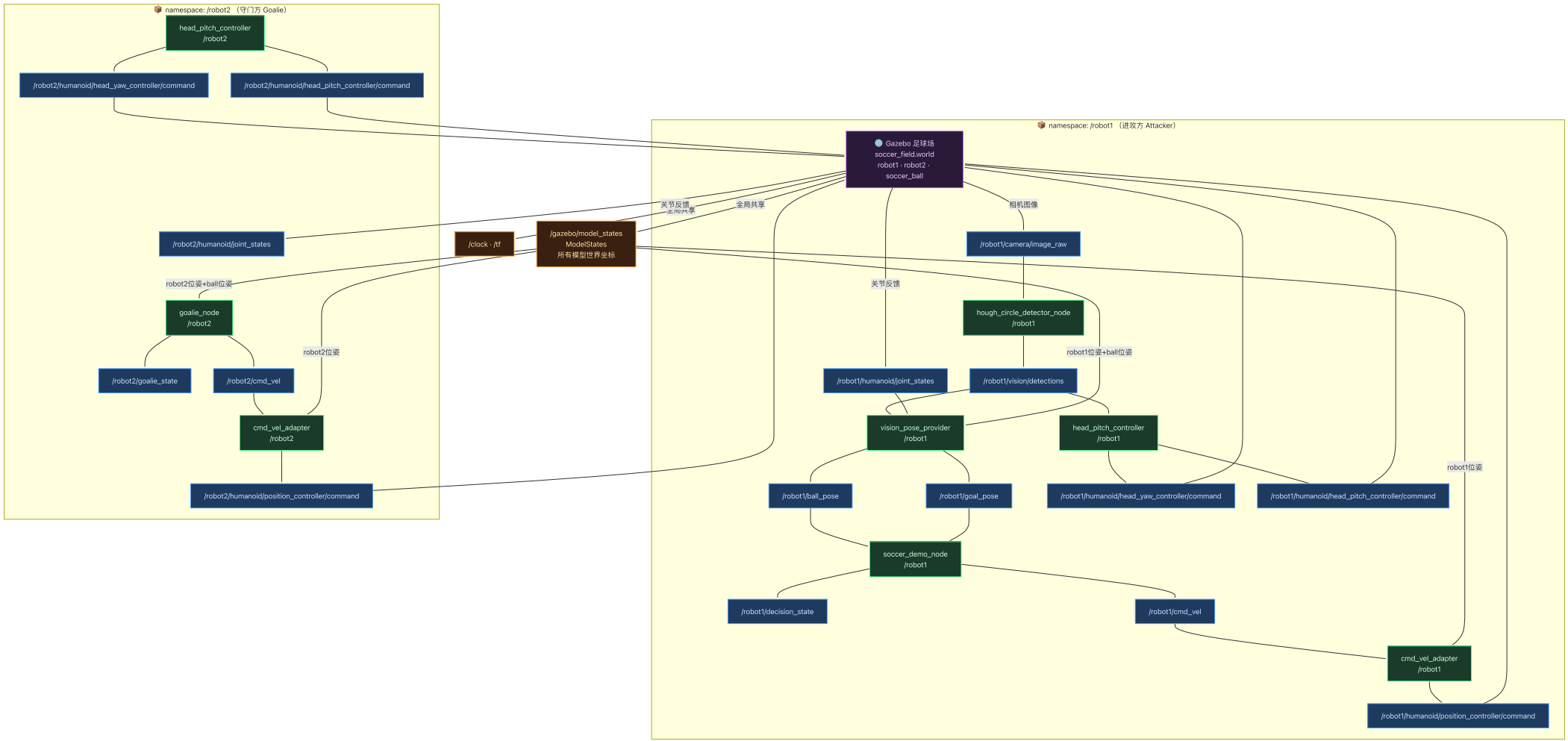
话题	类型	方向	说明
<code>/robot1/camera/image_raw</code>	<code>sensor_msgs/Image</code>	Gazebo→节点	头部相机图像
<code>/robot1/vision/detections</code>	<code>Float32MultiArray</code>	Hough→VPP	归一化检测结果
<code>/robot1/hough/debug_image</code>	<code>sensor_msgs/Image</code>	Hough→rqt	Hough 调试图
<code>/robot1/ball_pose</code>	<code>PoseStamped</code>	VPP→FSM	球相对机器人坐标
<code>/robot1/goal_pose</code>	<code>PoseStamped</code>	VPP→FSM	球门相对机器人坐标
<code>/robot1/decision_state</code>	<code>String</code>	FSM→监控	当前决策状态名
<code>/robot1/cmd_vel</code>	<code>Twist</code>	FSM→适配器	线速度+角速度指令
<code>/robot1/humanoid/joint_states</code>	<code>JointState</code>	Gazebo→VPP	头部关节角反馈
<code>/robot1/humanoid/position_controller/command</code>	<code>Float64MultiArray</code>	适配器→Gazebo	16 个关节角命令
<code>/robot1/humanoid/head_yaw_controller/command</code>	<code>Float64</code>	头部控制→Gazebo	头部水平转角
<code>/robot1/humanoid/head_pitch_controller/command</code>	<code>Float64</code>	头部控制→Gazebo	头部俯仰转角
<code>/robot1/reset_ball</code>	<code>std_srvs/Empty</code>	服务	重置球位置
<code>/robot1/reset_demo</code>	<code>std_srvs/Empty</code>	服务	重置机器人+球

I robot2（守门方）话题

话题	类型	方向	说明
<code>/robot2/goalie_state</code>	<code>String</code>	goalie→监控	<code>HOLD_GOAL</code> / <code>TRACK BALL_Y</code>
<code>/robot2/cmd_vel</code>	<code>Twist</code>	goalie→适配器	侧移+转向速度
<code>/robot2/humanoid/joint_states</code>	<code>JointState</code>	Gazebo→节点	头部关节角反馈
<code>/robot2/humanoid/position_controller/command</code>	<code>Float64MultiArray</code>	适配器→Gazebo	16 个关节角命令
<code>/robot2/humanoid/head_yaw_controller/command</code>	<code>Float64</code>	头部控制→Gazebo	头部水平转角
<code>/robot2/humanoid/head_pitch_controller/command</code>	<code>Float64</code>	头部控制→Gazebo	头部俯仰转角

话题	类型	说明
/gazebo/model_states	ModelStates	全场所有模型位姿（robot1/robot2/ball 都在里面）
/gazebo/set_model_state	服务	设置任意模型位置
/gazebo/pause_physics	服务	暂停物理引擎
/gazebo/unpause_physics	服务	恢复物理引擎
/clock	rosgraph_msgs/Clock	仿真时钟
/tf	tf2_msgs/TFMessage	TF 坐标变换树

三、双机器人通信结构图



四、为什么多机器人系统必须使用 Namespace

4.1 ROS Namespace 的本质

ROS 中所有话题、节点、参数都是全局字符串路径。没有 namespace 时，系统中所有节点共用一个平坦的命名空间，名字一旦相同就直接合并或覆盖。

Namespace 相当于给每个机器人的所有资源加了一层目录前缀，强制隔离不同机器人的信道。

无 namespace:	有 namespace:
/cmd_vel	/robot1/cmd_vel
/cmd_vel ← 冲突!	/robot2/cmd_vel ← 完全独立

4.2 本项目中 namespace 的三层作用

① 话题隔离（Topic Isolation）

```
<!-- launch 文件中的声明 -->
<group ns="robot1"> ... </group>
<group ns="robot2"> ... </group>
```

<group ns="robotX"> 内所有节点的相对话题名自动加前缀:

节点内写:	rospy.Publisher("cmd_vel", ...)
实际话题:	/robot1/cmd_vel 或 /robot2/cmd_vel

② 参数空间隔离（Parameter Isolation）

```
<!-- 同一份 demo_params.yaml 被加载两次，分别进入不同 namespace -->
<group ns="robot1">
  <rosparam file="demo_params.yaml" command="load"/>
</group>
<group ns="robot2">
  <rosparam file="demo_params.yaml" command="load"/>
</group>
```

```
/robot1/soccer_demo_node/approach_speed = 0.15
/robot2/soccer_demo_node/approach_speed = 0.15    ← 独立副本，互不影响
```

Ⓢ 控制器命名空间隔离（Controller Namespace）

ros_control 中控制器必须挂载在特定 namespace 下：

```
robot1 控制器： /robot1/humanoid/position_controller
robot2 控制器： /robot2/humanoid/position_controller
```

关节命令分别写入各自 namespace，Gazebo 插件根据 namespace 匹配对应机器人的关节。

Ⓢ TF 坐标树隔离（tf_prefix）

```
<param name="tf_prefix" value="robot1"/>
<param name="tf_prefix" value="robot2"/>
```

机器人各关节的 TF frame 名自动变为 **robot1/base_link**、**robot2/base_link**，避免 TF 树冲突。

五、没有 Namespace 时的冲突分析

5.1 冲突矩阵

假设去掉 **<group ns="robotX">**，所有话题退化为无前缀形式：

冲突话题	robot1 意图	robot2 意图	实际后果
/cmd_vel	FSM 输出"前进 0.15m/s"（进攻）	goalie 输出"侧移 0.08m/s"（防守）	两条消息同时到达 cmd_vel_adapter ，以随机到达顺序覆盖，两台机器人行为均混乱
/humanoid/position_controller/command	robot1 的腿部关节序列	robot2 的腿部关节序列	Gazebo 不知道命令给谁，可能两台机器人都收到混合的关节角，同时摔倒
/humanoid/joint_states	robot1 头部关节反馈	robot2 头部关节反馈	vision_pose_provider 用错误的头部角度计算球位置，定位完全失效
/vision/detections	robot1 相机看到的球	robot2 相机看到的球（若有）	FSM 可能根据 robot2 视角的检测结果决策 robot1 的行动
/decision_state	APPROACH_BALL	HOLD_GOAL	两者互相覆盖，监控无法区分哪台机器人处于什么状态
/humanoid/head_yaw_controller/command	robot1 头部朝向球	robot2 头部保持正前方	两台机器人头部争抢同一控制器命令，高频抖动
/robot_description	robot1 的 URDF	robot2 的 URDF	后加载的覆盖先加载的，robot_state_publisher 解析错误的模型

5.2 最严重的冲突场景还原

```
无 namespace 时的 cmd_vel 冲突时序：

t=0.00s  robot1 FSM 发布 cmd_vel: linear.x=0.15 (向球靠近)
         → cmd_vel_adapter 收到，robot1 和 robot2 都执行前进

t=0.05s  robot2 goalie 发布 cmd_vel: linear.y=0.08 (侧移跟球)
         → cmd_vel_adapter 收到，robot1 和 robot2 都执行侧移

t=0.10s  robot1 FSM 再次发布 cmd_vel: angular.z=0.3 (转向球)
         → robot1 和 robot2 都开始旋转

结果：
  robot1（进攻方）：画出不规则轨迹，无法靠近球
  robot2（守门方）：随机运动，完全离开球门线
  两台机器人均失控，相互干扰
```

5.3 参数冲突示例

```
无 namespace 加载两次 demo_params.yaml 时：

第一次加载（robot1）：
  /soccer_demo_node/approach_distance = 0.75

第二次加载（robot2）覆盖：
  /soccer_demo_node/approach_distance = 0.75    ← 相同值，此例无影响

但如果 robot1 和 robot2 需要不同参数：
  robot1 approach_speed = 0.15 (进攻需要快)
  robot2 approach_speed = 0.08 (守门只需侧移)
```


- 无 namespace 时无法为两个机器人配置不同参数
- 只能使用同一套参数，丧失灵活性

六、本项目 Namespace 实现的技术细节

6.1 节点内部处理相对/绝对路径

```
# soccer_demo_node.py (在 /robot1 namespace 下运行)
# 相对话题名 → 自动加 namespace 前缀
self.cmd_pub = rospy.Publisher("cmd_vel", Twist, ...)
# 实际发布到: /robot1/cmd_vel

self.ball_sub = rospy.Subscriber("ball_pose", ...)
# 实际订阅: /robot1/ball_pose

# 绝对路径 → 不受 namespace 影响 (跨机器人共享)
self.model_sub = rospy.Subscriber("/gazebo/model_states", ...)
# 永远是: /gazebo/model_states (两台机器人共同订阅)
```

6.2 Namespace 下参数的读取方式

```
# vision_pose_provider.py
# ~robot_model 读取私有参数 (~=节点私有 namespace)
self.robot_model = rospy.get_param("~robot_model", "robot1")
# launch 中通过 <param> 注入正确的值:
# <param name="robot_model" value="robot1"/> 或 value="robot2"
```

6.3 launch 文件中的精确控制

```
<!-- 绝对路径参数 (绕过 namespace) -->
<node>
  <param name="command_topic" value="/robot1/humanoid/position_controller/command"/>
</node>
<!-- 此处用绝对路径, 确保跨 namespace 时准确指向 -->
```

七、总结

Namespace 解决的核心问题:

	同一套代码 (cmd_vel_adapter / soccer_demo_node)	
	同一个 ROS Master	
	同时运行多个机器人实例	
	靠 namespace 前缀:	
	/robot1/* ↔ /robot2/*	
	完全隔离, 互不干扰	
	靠绝对路径 (/gazebo/*):	
	跨越 namespace 共享 Gazebo 全局状态	

工程意义:

- ✅ 代码零修改, 一套节点代码同时支持 N 台机器人
- ✅ 同一 yaml 配置文件加载到不同 namespace, 各自独立
- ✅ 监控调试时话题名清晰区分 (/robot1/ vs /robot2/)
- ✅ 可扩展到 robot3、robot4...只需复制 <group ns="...">

提高作业提交要求

请将提高作业整理为 一份 PDF 提交。